

Claude Agent SDK

Agent Loop Engineering

Beginner → Intermediate → Advanced

A step-by-step tutorial with real-world startup operation examples

Part 1 What Is an Agent Loop? — Core Concepts

Part 2 Installation & Your First Agent

Part 3 Turns, Messages & Result Handling

Part 4 Tools — Built-in & Custom

Part 5 Hooks — Control, Safety & Audit

Part 6 Sessions & 24/7 Continuity

Part 7 Multi-Agent Orchestration

Part 8 Real-World Startup Use Cases

Part 9 Production Patterns & Best Practices

Part 10 Workshop Exercises

Table of Contents

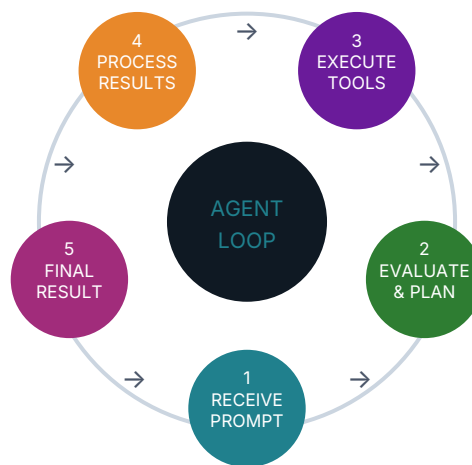
Part	Title	Page
Part 1	What Is an Agent Loop?	3
Part 2	Installation & Environment Setup	5
Part 3	Turns, Messages & Result Handling	8
Part 4	Tools — Built-in, Custom & Permissions	12
Part 5	Hooks — Control, Safety & Audit	17
Part 6	Sessions & 24/7 Continuity	23
Part 7	Multi-Agent Orchestration	27
Part 8	Real-World Startup Use Cases	32
Part 9	Production Patterns & Best Practices	44
Part 10	Workshop Exercises	49

PART 1 — What Is an Agent Loop?

BEGINNER

What Is an Agent Loop?

An agent loop is the repeating cycle that gives an AI the ability to think → act → observe → think again until a task is fully done. Without a loop, a language model answers once and stops. With a loop, it can read files, run commands, call APIs, check results, and keep going — autonomously — until the job is complete.



CONCEPT

Mental model: Think of the agent loop like a smart employee who has been given a task. They don't ask for permission on every small action — they use their tools (computer, phone, email), check progress, adjust their approach, and only report back when the task is truly done. The loop is the "thinking + doing" cycle that makes this possible.

Why Does the Loop Matter for Startups?

Startups operate with lean teams — often a handful of people doing the work of dozens. The agent loop enables one person to delegate entire operation workflows to AI. Here are the key gains:

Operation	Manual Effort/Week	With Agent Loop	Savings
Sprint planning & backlog	2–3 hours	15 minutes review	~85%
Customer support triage	4–6 hours	Fully automated	~90%
Code review & QA	3–5 hours	30 min oversight	~80%

Operation	Manual Effort/Week	With Agent Loop	Savings
Release notes & docs	2–3 hours	20 min review	~85%
Monitoring & alerts	1–2 hours	Continuous 24/7	~100%
Competitive research	2–4 hours	20 min review	~88%

Manual vs. Agent Loop — Side by Side

✖ MANUAL

Manual customer support: Engineer reads 50 support tickets every morning, categorises them, drafts replies, escalates 5 critical ones, updates Notion tracker. Takes 90 minutes per day.

✔ AGENT LOOP

Agent loop: Runs every 30 minutes. Reads new tickets, classifies priority, drafts & sends replies for L1 issues, creates Linear tickets for L2+, posts daily summary to Slack. Zero daily human time.

✖ MANUAL

Manual CI/CD: Developer merges PR, triggers build, watches logs, checks test results, reads error stack traces, manually notifies team in Slack if it fails. 20–40 min per broken build.

✔ AGENT LOOP

Agent loop: Triggered on merge. Runs tests, reads failure stack traces, searches codebase for root cause, drafts patch (optional), posts diagnosis to PR comment and Slack in under 5 minutes.

PART 2 — Installation & Environment Setup

BEGINNER

Installation & Environment Setup

Prerequisites

Item	Requirement	Check Command
Python	3.10 or higher	python3 --version
Node.js	18.x or higher (for TypeScript)	node --version
API Key	Anthropic account at platform.claude.com	—
Git	Any recent version	git --version

Install the SDK

```
BASH
# Python
pip install claude-agent-sdk

# TypeScript / Node.js
npm install @anthropic-ai/claude-agent-sdk
```

Set Your API Key

```
BASH
# On Linux / macOS
export ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxxxx

# On Windows (PowerShell)
$env:ANTHROPIC_API_KEY="sk-ant-xxxxxxxxxxxxxxxx"

# Or persist in .env file (never commit this!)
echo "ANTHROPIC_API_KEY=sk-ant-xxx" > .env
```

DANGER

Security: Never hard-code your API key in source code. Add .env to .gitignore immediately. Use environment variables or a secrets manager in production.

Your Very First Agent (5 lines)

This is the minimal working agent. It asks Claude to list files and prints the result.

PYTHON

```
import asyncio

from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage

async def main():
    async for message in query(
        prompt="What files are in the current directory?",
        options=ClaudeAgentOptions(allowed_tools=["Bash", "Glob"])
    ):
        if isinstance(message, ResultMessage) and message.subtype == "success":
            print(message.result)

asyncio.run(main())
```

TIP

Run it: Save as `hello_agent.py`, then run `python hello_agent.py`. You should see Claude list your directory contents. The agent took 1–2 turns to do this.

TypeScript Equivalent

TYPESCRIPT

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "What files are in the current directory?",
  options: { allowedTools: ["Bash", "Glob"] }
})) {
  if ("result" in message && message.subtype === "success") {
    console.log(message.result);
  }
}
```

PART 3 — Turns, Messages & Result Handling

BEGINNER

Turns, Messages & Result Handling

Understanding Turns

A turn is one full round trip inside the loop: Claude produces output including tool calls, the SDK executes those tools, and the results are fed back to Claude. A simple question might take 1–2 turns; a complex refactor might take 30+ turns. Only tool-use turns count toward `max_turns` — the final text-only reply does not.

Turn	What Claude Does	Tool Called	Result Back to Claude
1	Decides to read package.json	Read("package.json")	File contents with deps
2	Wants to run tests	Bash("npm test")	Test output: 3 failures
3	Reads failing test file	Read("auth.test.ts")	Test code
4	Reads source file	Read("auth.ts")	Source code with bug
5	Fixes the bug	Edit("auth.ts", patch)	Edit confirmed
6	Re-runs tests	Bash("npm test")	All 47 tests pass
Final	No tool calls — writes summary	—	ResultMessage delivered

The Five Message Types

Message Type	When Emitted	Key Fields	Use It For
SystemMessage	Session start & lifecycle events	subtype: "init" "compact_boundary"	Capture session_id for resume
AssistantMessage	After each Claude response	content (text + tool_use blocks)	See what Claude is doing in real-time
UserMessage	After each tool execution	content (tool_result blocks)	See tool outputs returned to Claude
StreamEvent	During streaming (if enabled)	Raw API streaming deltas	Show live typing effect in UI
ResultMessage	End of loop	subtype, result, cost, session_id	Get final answer + handle errors

Handling All Message Types — Complete Pattern

```
PYTHON
import asyncio
```

```
from claude_agent_sdk import (
    query, ClaudeAgentOptions,
    SystemMessage, AssistantMessage, UserMessage, ResultMessage
)

async def run_with_full_monitoring():
    session_id = None
    turn_count = 0

    async for msg in query(
        prompt="Analyze the codebase and identify the top 3 performance bottlenecks",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Grep", "Glob", "Bash"],
            max_turns=20,
            effort="high"
        )
    ):
        # 1. Capture session ID at start
        if isinstance(msg, SystemMessage) and msg.subtype == "init":
            session_id = msg.data.get("session_id")
            print(f"Session started: {session_id}")

        # 2. Track what Claude does each turn
        elif isinstance(msg, AssistantMessage):
            turn_count += 1

            for block in msg.content:
                if hasattr(block, "type") and block.type == "tool_use":
                    print(f"Turn {turn_count}: Using {block.name}")

        # 3. Handle final result
        elif isinstance(msg, ResultMessage):
            if msg.subtype == "success":
                print(f"\n=== RESULT ({msg.num_turns} turns) ===")
                print(msg.result)

                if msg.total_cost_usd:
                    print(f"Cost: ${msg.total_cost_usd:.4f}")

            elif msg.subtype == "error_max_turns":
                print(f"Hit turn limit. Resume: {session_id}")

            elif msg.subtype == "error_max_budget_usd":
```



```
print("Budget exceeded — reduce max_budget_usd")
else:
    print(f"Error: {msg.subtype}")

asyncio.run(run_with_full_monitoring())
```

Result Subtypes — What Each Means

Subtype	Meaning	result field?	Recommended Action
success	Task completed normally	Yes	Read message.result and proceed
error_max_turns	Hit turn limit	No	Resume session with higher max_turns
error_max_budget_usd	Hit cost limit	No	Resume or split into smaller tasks
error_during_execution	API failure / crash	No	Retry or check API status
error_max_structured_output_retries	Output validation failed	No	Check prompt clarity

PART 4 — Tools: Built-in, Custom & Permissions

INTERMEDIATE

Tools — Built-in, Custom & Permissions

Built-in Tools Reference

The SDK ships with every tool that powers Claude Code. These are available immediately — no extra setup required.

Tool	Category	What It Does	Parallel Safe?
Read	File	Read any file in working directory	Yes
Write	File	Create new files	No (sequential)
Edit	File	Make precise edits to existing files	No (sequential)
Bash	Execution	Run shell commands, scripts, git ops	No (sequential)
Monitor	Execution	Watch background script output line by line	Yes
Glob	Search	Find files by pattern (e.g. <code>**/*.py</code>)	Yes
Grep	Search	Search file contents with regex	Yes
WebSearch	Web	Search the web for current information	Yes
WebFetch	Web	Fetch and parse web page content	Yes
Agent	Orchestration	Spawn a subagent to handle a subtask	Yes
AskUserQuestion	Interaction	Pause loop and ask user a question	No
TaskCreate	Orchestration	Create a tracked background task	Yes
ToolSearch	Discovery	Find and load MCP tools on demand	Yes

Tool Permissions Deep Dive

Three settings work together to control what runs:

```
PYTHON
options = ClaudeAgentOptions(
    # 1. allowed_tools: these auto-approve without any prompt
    allowed_tools=["Read", "Glob", "Grep"],

    # 2. disallowed_tools: these NEVER run, regardless of anything else
    disallowed_tools=["Bash"],
```

```
# 3. permission_mode: what happens to everything else
# "default" — unlisted tools prompt your canUseTool callback
# "acceptEdits" — auto-approves file edits + mkdir/touch/mv/cp
# "plan" — explores but never auto-approves file edits
# "dontAsk" — denies everything not in allowed_tools
# "bypassPermissions" — allows ALL (containers/CI only!)
permission_mode="acceptEdits"
)
```

Scoped Bash Rules (Fine-Grained Control)

You can allow only specific Bash commands using rule syntax:

```
PYTHON
# Allow only git and npm commands
options = ClaudeAgentOptions(
    allowed_tools=[
        "Read", "Glob", "Grep",
        "Bash(git *)", # only git subcommands
        "Bash(npm *)", # only npm subcommands
        "Bash(pytest *)", # only pytest
    ],
    disallowed_tools=["Bash(rm -rf *)"], # never allow destructive rm
)
```

Defining a Custom Tool

Custom tools let your agent interact with any external system — your database, CRM, Slack, billing system, or internal API. Define them with a JSON Schema input spec and an async handler function.

```
PYTHON
from claude_agent_sdk import ClaudeToolDefinition, ClaudeAgentOptions, query

# --- Define the tool ---
fetch_customer_tool = ClaudeToolDefinition(
    name="FetchCustomer",
    description="Fetch customer data from CRM by email address. "
    "Returns name, plan, MRR, support ticket count, and churn risk score.",
```

```
input_schema={
    "type": "object",
    "properties": {
        "email": {
            "type": "string",
            "description": "Customer email address"
        }
    },
    "required": ["email"]
},
# readOnlyHint=True means this tool can run in parallel with other read-only tools
read_only_hint=True
)

# --- Implement the handler ---
async def handle_fetch_customer(tool_input: dict) -> str:
    email = tool_input["email"]
    # Replace with your actual CRM API call
    customer = await crm_client.get_customer(email=email)
    if not customer:
        return f"No customer found for {email}"
    return (
        f"Name: {customer.name}\n"
        f"Plan: {customer.plan} | MRR: ${customer.mrr}\n"
        f"Open tickets: {customer.open_tickets}\n"
        f"Churn risk: {customer.churn_risk_score:.1%}"
    )

# --- Wire it into the agent ---
async def run():
    async for msg in query(
        prompt="Analyse the churn risk for customers who raised tickets this week "
        "and draft a personalised outreach email for each high-risk customer.",
        options=ClaudeAgentOptions(
            custom_tools=[fetch_customer_tool],
            custom_tool_handler=handle_fetch_customer,
            allowed_tools=["Read", "WebSearch"],
            max_turns=30,
        )
    )
```

```
):  
if hasattr(msg, "result"):   
    print(msg.result)
```

TIP

Tool description quality matters enormously. Claude decides whether to call your tool based entirely on its description. Write descriptions that explain: (1) WHAT the tool does, (2) WHEN to use it, (3) what the output looks like. Poor descriptions = tool never called.

PART 5 — Hooks: Control, Safety & Audit

INTERMEDIATE

Hooks — Control, Safety & Audit

Hooks are callback functions that run at specific points in the agent loop. They are your primary mechanism for safety, compliance, and observability. Hooks run in your application process — not inside Claude's context window — so they don't consume tokens.

All Available Hook Events

Hook Event	Python	TS	Fires When	Common Use
PreToolUse	Yes	Yes	Before a tool executes	Block dangerous commands, validate inputs
PostToolUse	Yes	Yes	After tool returns result	Audit logs, trigger side effects
PostToolUseFailure	Yes	Yes	Tool execution fails	Error handling, alerting
UserPromptSubmit	Yes	Yes	Prompt sent to Claude	Inject extra context
Stop	Yes	Yes	Agent finishes	Save state, send summary notification
SubagentStart	Yes	Yes	Subagent spawned	Track parallel workloads
SubagentStop	Yes	Yes	Subagent completes	Aggregate parallel results
PreCompact	Yes	Yes	Context window compaction	Archive full transcript
Notification	Yes	Yes	Agent status messages	Forward to Slack/PagerDuty
PermissionRequest	Yes	Yes	Tool needs permission	Custom approval UI
SessionStart	No	Yes	Session initialised	Setup telemetry
SessionEnd	No	Yes	Session ends	Cleanup resources
PostToolBatch	No	Yes	Full batch of tools resolve	Inject conventions once

PreToolUse — Block Dangerous Operations

The most critical hook for production agents. Fires before EVERY tool call, giving you a chance to inspect inputs and block harmful operations.

```
PYTHON
import asyncio

from claude_agent_sdk import query, ClaudeAgentOptions, HookMatcher, ResultMessage

# --- Production safety: block destructive operations ---
async def production_safety_gate(input_data, tool_use_id, context):
```

```
hook_event = input_data.get("hook_event_name", "")
if hook_event != "PreToolUse":
    return {}

tool_name = input_data.get("tool_name", "")
tool_input = input_data.get("tool_input", {})

# Block dangerous shell patterns
if tool_name == "Bash":
    cmd = tool_input.get("command", "")
    DANGEROUS = ["rm -rf", "DROP TABLE", "DELETE FROM",
"> /dev/", "dd if=", "mkfs"]
    for pattern in DANGEROUS:
        if pattern in cmd:
            return {
                "systemMessage": f"Blocked dangerous command: {pattern}",
                "hookSpecificOutput": {
                    "hookEventName": hook_event,
                    "permissionDecision": "deny",
                    "permissionDecisionReason": (
                        f"Command contains dangerous pattern: {pattern}"
                    )
                }
            }

# Block writes to protected directories
if tool_name in ("Write", "Edit"):
    path = tool_input.get("file_path", "")
    PROTECTED = ["/etc/", "/usr/", "production.db", ".env"]
    for protected in PROTECTED:
        if protected in path:
            return {
                "hookSpecificOutput": {
                    "hookEventName": hook_event,
                    "permissionDecision": "deny",
                    "permissionDecisionReason": f"Path {path} is protected"
                }
            }
```

```

return {} # allow everything else

async def main():
    async for msg in query(
        prompt="Clean up old log files and optimise the database",
        options=ClaudeAgentOptions(
            allowed_tools=["Bash", "Read", "Edit", "Glob"],
            hooks={
                "PreToolUse": [HookMatcher(hooks=[production_safety_gate])]
            }
        )
    ):
        if isinstance(msg, ResultMessage):
            print(msg.result if msg.subtype == "success" else f"Error: {msg.subtype}")

    asyncio.run(main())

```

PostToolUse — Audit Logging

Fires after every tool call. Perfect for compliance logging without slowing the agent down (use async mode for non-blocking logging).

```

PYTHON

import asyncio, json
from datetime import datetime
from claude_agent_sdk import query, ClaudeAgentOptions, HookMatcher

AUDIT_LOG = "/var/log/agent_audit.jsonl"

async def audit_logger(input_data, tool_use_id, context):
    """Non-blocking audit log: agent continues immediately."""
    entry = {
        "ts": datetime.utcnow().isoformat(),
        "session": input_data.get("session_id"),
        "tool": input_data.get("tool_name"),
        "input": str(input_data.get("tool_input", {}))[:300],
        "result": str(input_data.get("tool_response", {}))[:300],
        "tool_id": tool_use_id,
    }
    with open(AUDIT_LOG, "a") as f:

```



```

f.write(json.dumps(entry) + "\n")

# Return async=True so agent continues without waiting
return {"async_": True, "asyncTimeout": 5000}

async def main():
    async for msg in query(
        prompt="Migrate user data to the new schema",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Bash", "Edit"],
            hooks={
                "PostToolUse": [HookMatcher(hooks=[audit_logger])]
            }
        )
    ):
        pass

    asyncio.run(main())

```

Stop Hook — Session Summary & Notifications

```

PYTHON

async def on_stop(input_data, tool_use_id, context):
    """Fires when agent finishes. Send summary to Slack."""
    import httpx

    summary = input_data.get("stop_reason", "completed")
    session = input_data.get("session_id", "")

    payload = {
        "text": f"*Agent session complete* `{session[:8]}`\n",
        "Status": {summary}\n",
        "Time": {datetime.utcnow().strftime('%H:%M UTC')},
        "channel": "#ops-alerts"
    }

    async with httpx.AsyncClient() as client:
        await client.post(SLACK_WEBHOOK_URL, json=payload)

    return {}

```

```
options = ClaudeAgentOptions(  
  hooks={"Stop": [HookMatcher(hooks=[on_stop])]}  
)
```

PART 6 — Sessions & 24/7 Continuity

INTERMEDIATE

Sessions & 24/7 Continuity

Every agent run creates a session — a persistent conversation with its own context history. You can resume a session to pick up exactly where you left off, or fork it to explore a different approach without losing the original.

Capturing and Resuming a Session

PYTHON

```
import asyncio, json, os

from claudesdk import query, ClaudeAgentOptions, SystemMessage, ResultMessage

SESSION_FILE = "/tmp/agent_session.json"

async def run_with_resume():
    # Load previously saved session (if any)
    saved = {}
    if os.path.exists(SESSION_FILE):
        with open(SESSION_FILE) as f:
            saved = json.load(f)

    session_id = saved.get("session_id")
    resume_opts = {}
    if session_id:
        resume_opts["resume"] = session_id
        print(f"Resuming session {session_id[:12]}...")
    else:
        print("Starting new session...")

    async for msg in query(
        prompt="Continue analysing the Q2 customer churn data — focus on the "
        "enterprise segment we found last time.",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Bash", "Glob", "WebSearch"],
            max_turns=25,
```

```

**resume_opts
)
):

if isinstance(msg, SystemMessage) and msg.subtype == "init":
    new_id = msg.data.get("session_id")
    # Persist session for next run
    with open(SESSION_FILE, "w") as f:
        json.dump({"session_id": new_id}, f)

if isinstance(msg, ResultMessage):
    print(msg.result if msg.subtype == "success" else f"Error: {msg.subtype}")

asyncio.run(run_with_resume())

```

Building a 24/7 Perpetual Monitoring Loop

This pattern runs an agent indefinitely, resuming the same session each cycle. Perfect for production monitoring, scheduled analysis, and background processing.

```

PYTHON

import asyncio, json, os, time

from claude_agent_sdk import query, ClaudeAgentOptions, SystemMessage, ResultMessage

SESSION_FILE = "/var/run/ops_agent_session.json"
POLL_INTERVAL = 300 # 5 minutes

def load_session():
    if os.path.exists(SESSION_FILE):
        with open(SESSION_FILE) as f:
            return json.load(f).get("session_id")
    return None

def save_session(sid):
    with open(SESSION_FILE, "w") as f:
        json.dump({"session_id": sid, "updated": time.time()}, f)

async def run_monitoring_cycle(session_id: str | None) -> str | None:
    """Run one monitoring cycle, return new session_id."""
    new_session_id = None
    opts = {"resume": session_id} if session_id else {}

```

```

async for msg in query(
    prompt="""Run the following operations monitoring checklist:

    1. Check API error rate in the last 5 minutes (read from /var/log/api.log)
    2. Check disk usage (df -h) – alert if any partition >80%
    3. Check for any new CRITICAL entries in app logs
    4. Summarise findings in 3 bullet points

    If anything critical is found, call the SlackAlert tool immediately.""",
    options=ClaudeAgentOptions(
        allowed_tools=["Bash", "Read", "Glob"],
        max_turns=15,
        max_budget_usd=0.50, # cap cost per cycle
        effort="medium", # balanced for routine checks
        **opts
    )
):
    if isinstance(msg, SystemMessage) and msg.subtype == "init":
        new_session_id = msg.data.get("session_id")
        save_session(new_session_id)
    if isinstance(msg, ResultMessage) and msg.subtype == "success":
        print(f"[{time.strftime('%H:%M')}] Cycle complete: {msg.result[:120]}...")

    return new_session_id

async def main():
    session_id = load_session()
    print("Starting 24/7 operations monitor...")

    while True:
        try:
            session_id = await run_monitoring_cycle(session_id)
        except Exception as e:
            print(f"Cycle error: {e}. Continuing...")

        print(f"Next check in {POLL_INTERVAL}s...")
        await asyncio.sleep(POLL_INTERVAL)

    asyncio.run(main())

```

PART 7 — Multi-Agent Orchestration

ADVANCED

Multi-Agent Orchestration

When a task is large, complex, or has independent parts, you can spawn subagents — separate Claude instances with their own fresh context, tool sets, and specialist instructions. Subagents can run in parallel, dramatically cutting total runtime.

Key Benefits of Subagents

Benefit	How It Works	Example
Context Isolation	Each subagent starts fresh — intermediate work stays in its own window	Research agent reads 50 files; main agent sees only the summary
Parallelisation	Multiple subagents run concurrently	3 analysis agents finish in time of 1 instead of 3x
Specialisation	Each subagent has a focused system prompt & minimal tool set	Security agent knows CVE patterns; test agent knows pytest
Cost Control	Use cheaper models (Haiku/Sonnet) for routine subtasks	Main = Opus for strategy, subagents = Sonnet for execution

Defining and Using Subagents

PYTHON

```
import asyncio

from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition, ResultMessage


async def run_parallel_review():
    async for msg in query(
        # Main agent orchestrates; subagents do the work
        prompt="""Use all three specialist agents to review the new payment module:
        1. Use code-reviewer to check code quality and patterns
        2. Use security-scanner to find vulnerabilities
        3. Use test-runner to run and analyse existing tests
        Then synthesise findings into a single actionable report.""",
        options=ClaudeAgentOptions(
            # Agent tool must be in allowed_tools to auto-approve subagent invocations
            allowed_tools=["Read", "Grep", "Glob", "Bash", "Agent"],
            max_turns=40,
```

```
agents={
    "code-reviewer": AgentDefinition(
        description="Code quality reviewer. Use for style, patterns, maintainability.",
        prompt="""You are a senior engineer reviewing for code quality.
        Check: SOLID principles, naming clarity, error handling, logging, documentation.
        Be specific – cite file:line for every issue.""",
        tools=["Read", "Grep", "Glob"],
        model="sonnet", # cheaper model for routine review
        effort="medium"
    ),

    "security-scanner": AgentDefinition(
        description="Security vulnerability scanner. Use for auth, injection, secrets.",
        prompt="""You are an application security expert.
        Scan for: SQL/NoSQL injection, hardcoded secrets, broken auth, insecure deserialization,
        OWASP Top 10, and PCI-DSS issues in payment code. Report every finding with severity.""",
        tools=["Read", "Grep", "Glob"],
        model="opus", # use more capable model for security
        effort="high"
    ),

    "test-runner": AgentDefinition(
        description="Test execution specialist. Use for running tests and coverage.",
        prompt="""You run tests and analyse results.
        Execute all relevant test suites, report failures with stack traces,
        calculate coverage, and identify untested edge cases.""",
        tools=["Bash", "Read", "Grep"],
        model="sonnet",
    )
}

):
    if isinstance(msg, ResultMessage) and msg.subtype == "success":
        print(msg.result)

asyncio.run(run_parallel_review())
```

TIP

Parallel execution: When your prompt asks the main agent to invoke multiple subagents, Claude may call them in parallel (using multiple Agent tool calls in the same turn). This means 3 parallel 10-minute subagents finish in ~10 minutes, not 30. Design subagents to be independent for maximum parallelism.

AgentDefinition — Full Configuration Reference

Field	Required	Type	Description
description	Yes	string	Natural language — when Claude invokes this agent
prompt	Yes	string	System prompt defining role, expertise, constraints
tools	No	string[]	Allowed tools; inherits parent tools if omitted
disallowedTools	No	string[]	Tools to remove (MCP server patterns ok)
model	No	string	Model alias: "opus", "sonnet", "haiku", or full model ID
effort	No	string	"low" "medium" "high" "xhigh" "max"
maxTurns	No	number	Cap turns for this subagent specifically
background	No	boolean	Run as non-blocking background task
skills	No	string[]	Skill names to preload into subagent context
mcpServers	No	array	MCP servers available to this subagent
permissionMode	No	string	Permission mode override for this subagent

PART 8 — Real-World Startup Use Cases

INTERMEDIATE

Real-World Startup Use Cases

This section presents 6 complete, production-ready agent loop patterns that tackle the most time-consuming startup operations. Each example includes the full code, explanation of the design decisions, and expected output.

Use Case 1 — Customer Support Triage Agent

REAL WORLD

Context: A SaaS startup receives 50–200 support tickets per day via email and Intercom. The founding team was spending 3–4 hours daily reading, categorising, and responding to L1 tickets. This agent reduced that to a 10-minute daily review.

PYTHON

```
import asyncio

from claude_agent_sdk import (
    query, ClaudeAgentOptions, ClaudeToolDefinition, ResultMessage
)

# --- Custom tools connecting to your support systems ---
list_new_tickets_tool = ClaudeToolDefinition(
    name="ListNewTickets",
    description="Fetch unassigned support tickets created in the last N hours. "
    "Returns ticket ID, subject, customer plan, and body text.",
    input_schema={
        "type": "object",
        "properties": {
            "hours": {"type": "integer", "description": "Look-back window in hours"},
            "status": {"type": "string", "enum": ["new", "open", "pending"]}
        },
        "required": ["hours"]
    },
    read_only_hint=True
)

send_reply_tool = ClaudeToolDefinition(
```

```

name="SendTicketReply",
description="Send a reply to a support ticket and update its status. "
"For L1 issues: status=resolved. For L2+: status=escalated.",
input_schema={
  "type": "object",
  "properties": {
    "ticket_id": {"type": "string"},
    "reply": {"type": "string", "description": "Reply text in markdown"},
    "status": {"type": "string", "enum": ["resolved", "escalated", "waiting"]},
    "priority": {"type": "string", "enum": ["low", "normal", "high", "urgent"]}
  },
  "required": ["ticket_id", "reply", "status"]
}
)

TRIAGE_PROMPT = """
You are a customer support specialist for a B2B SaaS product.

Your workflow for each ticket:
1. Read ticket subject + body carefully
2. Classify: L1 (common/documented), L2 (requires engineering), L3 (critical/data loss)
3. For L1: Draft a warm, helpful reply using product knowledge. Mark as resolved.
4. For L2: Add context, draft initial reply, escalate to #eng-support in Slack.
5. For L3: Send to on-call engineer immediately via PagerDuty.

Tone: Professional, empathetic, solution-focused. Never say "I cannot help."
Always cite specific docs or steps when available.
"""

async def run_support_agent():
    async for msg in query(
        prompt="Process all new support tickets from the last 4 hours. "
        "Triage, respond to L1 issues, and escalate L2/L3 appropriately.",
        options=ClaudeAgentOptions(
            system_prompt=TRIAGE_PROMPT,
            custom_tools=[list_new_tickets_tool, send_reply_tool],
            custom_tool_handler=support_tool_handler, # your implementation
            allowed_tools=["WebSearch"], # allow docs search
            max_turns=50,

```

```

max_budget_usd=2.00, # cap at $2 per triage run
effort="medium",
)
):
if isinstance(msg, ResultMessage):
    print(f"Support run complete: {msg.result}")

# Schedule this to run every 30 minutes via cron or task queue
asyncio.run(run_support_agent())

```

TIP

Result: This pattern processes 50 tickets in under 4 minutes for roughly \$0.80–\$1.50 in API cost. That replaces 3–4 hours of founder time daily. At \$100/hour founder opportunity cost, ROI is 200:1.

Use Case 2 — CI/CD Failure Diagnosis Agent

REAL WORLD

Context: Every broken CI build used to cost 20–40 minutes of an engineer's time to diagnose. This agent fires on build failure webhooks, diagnoses root cause, and posts an actionable fix to the PR within 5 minutes.

PYTHON

```

import asyncio, os

from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage

async def diagnose_build_failure(build_id: str, repo_path: str, log_url: str):
    """
    Called from your CI webhook handler when a build fails.
    Posts diagnosis to GitHub PR and Slack.
    """
    async for msg in query(
        prompt=f"""A CI build has failed. Your job is to diagnose and help fix it.

        Build ID: {build_id}
        Repository: {repo_path}
        Failure logs: {log_url}

        Steps:
        1. Fetch the failure logs from {log_url} using WebFetch
        2. Read the relevant source files mentioned in the stack trace

```

```

3. Search the codebase for the root cause
4. Write a clear diagnosis: what failed, why, and how to fix it
5. If you can safely fix it (lint error, import issue, typo), write the fix
6. Output a GitHub-flavoured markdown comment ready to post to the PR
"""
options=ClaudeAgentOptions(
    allowed_tools=["WebFetch", "Read", "Grep", "Glob", "Bash", "Edit"],
    permission_mode="acceptEdits",
    max_turns=20,
    max_budget_usd=1.00,
    effort="high", # thorough reasoning for debugging
)
):
if isinstance(msg, ResultMessage) and msg.subtype == "success":
    # Post to GitHub PR and Slack
    await post_pr_comment(build_id, msg.result)
    await notify_slack(
        channel="#eng-builds",
        text=f"Build `{build_id}` failed. Diagnosis ready - "
        f"see PR comment. Cost: ${msg.total_cost_usd:.3f}"
    )
return msg.result

```

Use Case 3 — Sprint Planning Automation Agent

REAL WORLD

Context: Weekly sprint planning was taking 2–3 hours. Issues were ungroomed, estimates were missing, and prioritisation was inconsistent. This agent does the prep work, leaving humans 30 minutes for final decisions.

PYTHON

```

import asyncio

from claude_agent_sdk import (
    query, ClaudeAgentOptions, AgentDefinition, ResultMessage
)

SPRINT_ORCHESTRATOR_PROMPT = """
You are the Engineering Lead running sprint planning prep.
Coordinate the specialist agents below to:

1. Groom the backlog – break down epics, add acceptance criteria

```

```

2. Prioritise issues – score by impact × effort / technical risk
3. Estimate complexity – add story point estimates
4. Balance the sprint – ensure the sprint is achievable given team capacity
5. Draft the sprint doc – publish to Notion
Output a complete sprint plan ready for the team to review.
"""

async def run_sprint_planning(linear_project_id: str, team_capacity_pts: int):
    async for msg in query(
        prompt=f"""Run complete sprint planning for project {linear_project_id}.
        Team capacity this sprint: {team_capacity_pts} story points.
        Today: {__import__('datetime').date.today()}""",

        options=ClaudeAgentOptions(
            system_prompt=SPRINT_ORCHESTRATOR_PROMPT,
            allowed_tools=["Agent", "Read"],
            max_turns=30,

            agents={
                "backlog-groomer": AgentDefinition(
                    description="Grooms backlog items: breaks down vague issues, "
                    "adds acceptance criteria, identifies blockers.",
                    prompt="You groom engineering backlogs. For each issue: "
                    "split if >5 pts, add Given/When/Then acceptance criteria, "
                    "flag dependencies. Use Linear MCP to read and update issues.",
                    tools=["mcp__linear__list_issues", "mcp__linear__update_issue"],
                    model="sonnet",
                ),

                "prioritiser": AgentDefinition(
                    description="Prioritises issues using RICE scoring "
                    "(Reach × Impact × Confidence / Effort).",
                    prompt="Score each backlog issue on RICE. Read customer data and "
                    "product strategy docs to inform Reach and Impact scores. "
                    "Output a ranked list with scores and rationale.",
                    tools=["Read", "Glob", "mcp__linear__list_issues"],
                    model="opus", # needs strategic judgment
                    effort="high",
                ),
            },
        ),
    )

```

```

"estimator": AgentDefinition(
    description="Estimates story points for technical issues using "
    "codebase complexity analysis.",
    prompt="Estimate story points (1,2,3,5,8,13) for each issue. "
    "Read relevant source files to understand complexity. "
    "Consider: code changes needed, testing effort, risk.",
    tools=["Read", "Grep", "Glob"],
    model="sonnet",
),
}
):
    if isinstance(msg, ResultMessage) and msg.subtype == "success":
        print(msg.result)

asyncio.run(run_sprint_planning("PROJ-123", team_capacity_pts=40))

```

Use Case 4 — Weekly Competitive Intelligence Agent

REAL WORLD

Context: Keeping up with 8 competitors across their blogs, pricing pages, job listings, and GitHub repos was a 3-hour weekly task for the product team. This agent runs every Monday at 7am and posts a briefing to Slack.

PYTHON

```

import asyncio

from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition, ResultMessage

COMPETITORS = [
    {"name": "CompetitorA", "url": "https://competitora.io",
    "pricing": "https://competitora.io/pricing", "blog": "https://competitora.io/blog"},
    {"name": "CompetitorB", "url": "https://competitorb.com",
    "pricing": "https://competitorb.com/pricing", "blog": "https://competitorb.com/changelog"},
    # ... add more competitors
]

async def run_competitive_intelligence():
    competitor_list = "\n".join(
        f"- {c['name']}: {c['url']} | Pricing: {c['pricing']} | Blog: {c['blog']}"
    )

```

```

for c in COMPETITORS
)

async for msg in query(
prompt=f"""Conduct weekly competitive intelligence for these competitors:
{competitor_list}

For each competitor:

1. Visit their homepage and pricing page – note any changes from last week
2. Read their latest 2 blog posts or changelog entries
3. Search for news mentions in the last 7 days
4. Check their job board for signals (what roles are they hiring?)

Then synthesise:

- Top 3 strategic moves across all competitors this week
- Any pricing changes we need to respond to
- Feature gaps they're closing
- Recommended actions for our team

Format as a Slack-ready markdown message under 500 words.""",

options=ClaudeAgentOptions(
allowed_tools=["WebFetch", "WebSearch", "Agent"],
agents={
# One analyst per competitor runs in parallel
"competitor-analyst": AgentDefinition(
description="Analyses a single competitor's online presence and activity.",
prompt="You monitor a specific competitor. Fetch their key pages, "
"search for recent news, and produce a structured JSON report "
"with: changes_detected, pricing_update, key_announcements, "
"hiring_signals, threat_level (low/medium/high).",
tools=["WebFetch", "WebSearch"],
model="sonnet",
)
},
max_turns=40,
max_budget_usd=3.00,
)
):

```

```

if isinstance(msg, ResultMessage) and msg.subtype == "success":
    # Post to #product-intel Slack channel
    await post_to_slack("#product-intel", msg.result)

asyncio.run(run_competitive_intelligence())

```

Use Case 5 — Automated Release Notes Generator

REAL WORLD

Context: Writing release notes from git commits and PR descriptions was taking 2 hours per release. This agent runs on every GitHub release tag and auto-generates structured release notes for the changelog and marketing team.

PYTHON

```

import asyncio

from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage

async def generate_release_notes(version: str, prev_version: str):
    async for msg in query(
        prompt=f"""Generate release notes for version {version} (from {prev_version}).

Steps:
1. Run: git log {prev_version}..{version} --oneline --pretty=format:"%h %s %an"
2. For each significant commit, read the PR description (via Bash gh pr view)
3. Group changes into: ✨ New Features, 🐛 Bug Fixes, ⚡ Performance,
   🔒 Security, ⚠ Breaking Changes, 🏠 Internal
4. Write customer-facing release notes (non-technical language)
5. Write technical changelog (for developers)
6. Write a 2-sentence marketing summary

Output all three sections clearly separated.""",

        options=ClaudeAgentOptions(
            allowed_tools=[
                "Bash(git *)",
                "Bash(gh *)", # GitHub CLI
                "Read", "Grep"
            ],
            permission_mode="dontAsk", # fully autonomous
            max_turns=25,

```



```

    effort="medium",
)
):
    if isinstance(msg, ResultMessage) and msg.subtype == "success":
        # Commit to CHANGELOG.md, update GitHub release, post to Slack
        await update_github_release(version, msg.result)
        return msg.result

asyncio.run(generate_release_notes("v2.4.0", "v2.3.0"))

```

Use Case 6 — Customer Onboarding Agent

REAL WORLD

Context: New customer onboarding required an engineer to review their environment, check integration health, and write a customised success plan. This took 45–90 minutes per customer. The agent does it in under 10 minutes.

PYTHON

```

import asyncio

from claude_agent_sdk import query, ClaudeAgentOptions, ClaudeToolDefinition, ResultMessage

check_integration_tool = ClaudeToolDefinition(
    name="CheckIntegrationHealth",
    description="Check if a customer's integration is correctly configured. "
    "Tests API connectivity, webhook delivery, and data flow. "
    "Returns: status, error_details, first_event_received, setup_completeness_pct",
    input_schema={
        "type": "object",
        "properties": {
            "customer_id": {"type": "string"},
            "integration": {"type": "string",
                "enum": ["stripe", "salesforce", "hubspot", "jira", "slack"]}
        },
        "required": ["customer_id", "integration"]
    },
    read_only_hint=True
)

async def run_onboarding_agent(customer_id: str, customer_name: str,
    plan: str, integrations: list[str]):

```

```
integrations_list = ", ".join(integrations)

async for msg in query(
    prompt=f"""Run new customer onboarding for {customer_name} (ID: {customer_id}).
    Plan: {plan} | Integrations requested: {integrations_list}

    1. Check health of all requested integrations
    2. Identify any setup issues or missing configurations
    3. Search our knowledge base for integration-specific best practices
    4. Generate a personalised success plan with:
    - Week 1: Quick wins and immediate value
    - Week 2-4: Core workflow setup
    - Month 2-3: Advanced features and optimisation
    5. Draft a welcome email from their assigned CSM
    6. Create onboarding tasks in Linear for the CS team

    Be specific to their plan ({plan}) and industry context.""",
    options=ClaudeAgentOptions(
        custom_tools=[check_integration_tool],
        custom_tool_handler=integration_handler,
        allowed_tools=["WebSearch", "Read", "Glob"],
        max_turns=30,
        effort="high", # thorough for customer-facing output
    )
):
    if isinstance(msg, ResultMessage) and msg.subtype == "success":
        print(f"Onboarding plan ready for {customer_name}")
        return msg.result

asyncio.run(run_onboarding_agent(
    customer_id="cust_abc123",
    customer_name="Acme Corp",
    plan="enterprise",
    integrations=["stripe", "salesforce", "slack"]
))
```

PART 9 — Production Patterns & Best Practices

ADVANCED

Production Patterns & Best Practices

Cost Control — The Budget Framework

Unconstrained agent loops can run many turns and accumulate significant cost. Always set cost controls in production:

PYTHON

```
# Recommended production cost controls

options = ClaudeAgentOptions(
    # Hard turn limit - prevents infinite loops
    max_turns=30,

    # Hard dollar limit - stops loop when budget hit
    max_budget_usd=2.00,

    # Choose the right effort level:
    # "low" - file lookups, listing, grep (~0.5x cost)
    # "medium" - routine tasks, standard edits (1x cost)
    # "high" - debugging, refactoring (1.5-2x cost)
    # "xhigh" - complex analysis (3-4x cost)
    # "max" - deepest problems only (5x+ cost)
    effort="medium",

    # Use cheaper model for subagents
    agents={
        "routine-worker": AgentDefinition(
            description="...",
            prompt="...",
            model="sonnet", # vs opus for main agent
            effort="low", # minimal reasoning for simple tasks
            max_turns=10 # cap subagent turns too
        )
    }
)
```

Effort Level vs Cost vs Quality Trade-off

Effort Level	Relative Cost	Best For	Avoid For
low	~0.5x	File listing, grep, simple lookups	Debugging, security analysis
medium	1x (baseline)	Routine edits, standard tasks	Complex refactors
high	~2x	Debugging, code review	Simple file operations
xhigh	~4x	Architecture decisions, Opus 4.7+	Routine operations
max	~6x+	Multi-step deep analysis only	Everything routine

Context Window Management

MARKDOWN

```
# CLAUDE.md – put project-critical rules here, not in the prompt.
# CLAUDE.md is re-injected on every request and survives compaction.

# CLAUDE.md example for a production agent:
"""

# Agent Rules

## Safety Rules (NEVER violate)
- Never modify files in /prod/ or /etc/
- Never delete data without explicit user confirmation
- Never expose API keys in output

## When summarising conversations, always preserve:
- Current task objective and acceptance criteria
- All file paths that have been read or modified
- Error messages and stack traces encountered
- Decisions made and reasoning
- Open action items

## Tool preferences
- Always run tests after editing code
- Use grep before read for searching large codebases
- Prefer read-only tools; confirm before writes
"""
```

Error Handling & Retry Pattern

PYTHON

```
import asyncio, time

from claudesdk import query, ClaudeAgentOptions, ResultMessage

async def robust_agent_run(prompt: str, max_retries: int = 3) -> str | None:
    """Agent runner with exponential backoff and session resume on turn-limit hits."""
    session_id = None
    remaining_prompt = prompt

    for attempt in range(max_retries):
        try:
            async for msg in query(
                prompt=remaining_prompt,
                options=ClaudeAgentOptions(
                    allowed_tools=["Read", "Bash", "Glob"],
                    max_turns=25,
                    max_budget_usd=3.00,
                    **({"resume": session_id} if session_id else {})
                )
            ):
                if isinstance(msg, ResultMessage):
                    session_id = msg.session_id

                    if msg.subtype == "success":
                        return msg.result

                    elif msg.subtype == "error_max_turns":
                        # Resume in next attempt with continuation prompt
                        remaining_prompt = (
                            "Continue where you left off. The previous attempt hit the "
                            "turn limit. Complete the remaining work."
                        )
                        print(f"Turn limit hit – resuming (attempt {attempt+1})")
                        break

                    elif msg.subtype == "error_during_execution":
                        wait = 2 ** attempt
                        print(f"Execution error – retry in {wait}s")
                        await asyncio.sleep(wait)
```

```
break

else:
    print(f"Terminal error: {msg.subtype}")
    return None

except Exception as e:
    wait = 2 ** attempt
    print(f"Unexpected error: {e} - retry in {wait}s")
    await asyncio.sleep(wait)

return None # all retries exhausted
```

Production Deployment Checklist

Item	Why	Done?
Set max_turns (≤50)	Prevent infinite loops	
Set max_budget_usd	Cap API spend per run	
Add PreToolUse safety hook	Block dangerous ops	
Add PostToolUse audit log	Compliance & debugging	
Persist session_id to disk	Enable resume after crash	
Wrap in try/except with retry	Handle transient API errors	
Set ANTHROPIC_API_KEY via env var	Never hardcode secrets	
Use permission_mode="dontAsk" in CI	Unattended production runs	
Use "bypassPermissions" only in containers	Isolation required	
Set effort=low/medium for routine tasks	Cost optimisation	
Use Sonnet/Haiku models for subagents	Cost vs quality balance	
Add CLAUDE.md to project root	Rules survive compaction	
Test with small max_turns first	Validate before scaling	

PART 10 — Workshop Exercises

ALL LEVELS

Workshop Exercises

Complete these exercises in order. Each builds on the previous. Expected total time: 3–4 hours.

Exercise 1 Hello Agent Loop BEGINNER • 15 min

Goal: Run your first agent and observe all message types.

Task:

Write an agent that:

1. Lists all Python files in the current directory
2. Reads the largest one
3. Counts the number of functions (def ...) in it
4. Prints the result with turn count and cost

Then add turn tracking: print "Turn N: [tool called]" for each AssistantMessage.

Hints: Use Glob to find files, Bash(wc) or Grep to count, ResultMessage for cost.

Exercise 2 Custom Tool + Handler BEGINNER • 25 min

Goal: Build and wire a custom tool to an external data source.

Task:

Create a "FetchWeather" custom tool that:

1. Takes city_name as input
2. Calls wttr.in API: <https://wttr.in/{city}?format=j1> (free, no key needed)
3. Returns temperature, conditions, humidity

Wire it to an agent that: reads a CSV file of city names, fetches weather for each, and produces a travel advisory report ranking cities by comfort score.

Hints: Use httpx or urllib for the API call. Set read_only_hint=True.

Exercise 3 Safety Hook Engineering INTERMEDIATE • 30 min

Goal: Build a PreToolUse hook that enforces file write policies.

Task:

Build a PreToolUse hook that:

1. Blocks any Write/Edit to files matching *.prod.*, *.env, *secrets*
2. Requires files in /tmp/ to have a .sandbox_ prefix (redirect automatically using updatedInput)
3. Logs every blocked attempt to blocked_attempts.jsonl
4. Sends a mock Slack alert for 3+ blocks in a single session

Test it by asking the agent to "update the production config and create some temp files."

Hints: Use updatedInput to redirect paths. Use session-level state (a dict) to count blocks.

Exercise 4 Session Resume & Continuity

INTERMEDIATE • 30 min

Goal: Build a multi-turn analysis that resumes across restarts.

Task:

Build a code analysis agent that:

1. On first run: reads all Python files in a repo, identifies TODO comments, saves session_id to disk, stops after 10 turns
2. On second run: resumes session, continues analysis, produces final report
3. The final report should reference findings from BOTH runs

Verify by killing the process after the first run and re-running manually.

Hints: Save session_id to a JSON file. Use ClaudeAgentOptions(resume=session_id).

Exercise 5 Multi-Agent Code Review Pipeline

ADVANCED • 45 min

Goal: Build a parallel 3-agent code review system.

Task:

Build a code review orchestrator with 3 specialist subagents:

- style-checker: checks PEP8, naming, docstrings (model=sonnet, effort=low)
- logic-reviewer: analyses algorithms, edge cases, bugs (model=opus, effort=high)
- security-scanner: finds vulnerabilities, hardcoded secrets (model=opus, effort=high)

The main agent: receives a file path, spawns all 3 in parallel, waits for results, synthesises a single review with severity ratings (INFO / WARNING / CRITICAL), and writes the review to {filename}.review.md

Track when each subagent starts/stops using SubagentStart/SubagentStop hooks.

Hints: Include "Agent" in allowed_tools. Subagents run in parallel when Claude calls them in the same turn.

Exercise 6 Production Ops Agent

ADVANCED • 50 min

Goal: Build a complete operations monitoring agent.

Task:

Build a 24/7 operations agent that:

1. Runs every 5 minutes (use `asyncio.sleep` or integrate with a cron library)
2. Checks: disk usage (`df`), process count (`ps aux | wc -l`), log errors (`tail -100 /var/log/syslog`)
3. Maintains a session across runs (resume pattern)
4. Has a safety hook blocking any Bash command longer than 200 chars
5. Sends a mock Slack summary every 30 minutes (use a Stop hook counter)
6. Handles the `error_max_turns` case gracefully and resumes

Bonus: Add a `FetchMetrics` custom tool that simulates reading from a time-series DB.

Hints: Store cycle count in a file between runs. Use Stop hook to detect end of each cycle.

QUICK REFERENCE — Agent Loop Cheat Sheet

Agent Loop Cheat Sheet

Minimal Working Agent

```
PYTHON

from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage
import asyncio

async def run():
    async for msg in query(
        prompt="Your task here",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Bash", "Glob"],
            max_turns=20,
            max_budget_usd=1.00,
        )
    ):
        if isinstance(msg, ResultMessage) and msg.subtype == "success":
            print(msg.result)

asyncio.run(run())
```

Key Options at a Glance

Option (Python)	Type	Effect
allowed_tools	list[str]	Auto-approve listed tools without prompting
disallowed_tools	list[str]	Block listed tools absolutely
permission_mode	str	"default" "acceptEdits" "dontAsk" "bypassPermissions"
max_turns	int	Max tool-use round trips (final text turn not counted)
max_budget_usd	float	Stop loop when this USD cost is reached
effort	str	"low" "medium" "high" "xhigh" "max"
model	str	Model ID or alias ("sonnet", "opus", "haiku")
resume	str	session_id to resume a previous session
system_prompt	str	Override the agent system prompt
custom_tools	list	ClaudeToolDefinition objects for external APIs
hooks	dict	Event → list[HookMatcher] for control & observability

Option (Python)	Type	Effect
agents	dict	Name → AgentDefinition for multi-agent setups
mcp_servers	dict	Name → server config for MCP integrations
setting_sources	list	["project", "global"] — where to load CLAUDE.md from

Hook Return Values

Return Field	Effect
{}	Allow operation, no changes
"permissionDecision": "deny"	Block the tool call
"permissionDecision": "allow"	Auto-approve (use with updatedInput)
"updatedInput": {...}	Replace tool input before execution
"additionalContext": "..."	Append text to tool result (PostToolUse only)
"updatedToolOutput": "..."	Replace tool result before Claude sees it
"systemMessage": "..."	Show message to user (does not affect Claude)
"continue_": False	Stop the agent after this hook
"async_": True	Return immediately, run hook in background

NOTE

Further reading: Agent Loop: code.claude.com/docs/en/agent-sdk/agent-loop | Hooks: code.claude.com/docs/en/agent-sdk/hooks | Subagents: code.claude.com/docs/en/agent-sdk/subagents | Sessions: code.claude.com/docs/en/agent-sdk/sessions | Custom Tools: code.claude.com/docs/en/agent-sdk/custom-tools